



Capítulo 6 - Identificação da posição dos trens

A sequência de passagens ajuda a acompanhar o trem, desde que o programa conheça os limites dessa informação.

Introdução	No capítulo anterior, lemos vários sensores e reconhecemos suas mudanças de estado. Agora vamos utilizar os eventos de passagem para acompanhar a localização estimada de um trem.
Objetivo	Relacionar a passagem por sensores a uma sequência conhecida, registrar o último ponto detectado e reconhecer situações em que a posição permanece desconhecida.
Meta de aprendizagem	Ao final, você deverá conseguir acompanhar uma sequência simples de um único trem, distinguir detecção e estimativa e tratar uma passagem inesperada.

1. Detectar um ponto não é localizar todo o trem

Um sensor Hall detecta a aproximação de um ímã em um ponto da ferrovia. Se o ímã estiver sob a locomotiva, o acionamento informa a passagem dessa parte da composição pelo sensor.

Depois que o ímã se afasta, o sensor deixa de ser acionado. Isso não informa se os vagões já passaram nem se todo um trecho ficou livre.

A mensagem “passagem detectada em S11” é uma observação. A informação “trem seguindo de S11 para S12” é uma estimativa baseada na sequência prevista e nas condições de operação.

O programa não obtém uma coordenada contínua ao longo dos trilhos. Entre dois sensores, utiliza o histórico de passagens para acompanhar uma região estimada.

2. Condições do exercício

Vamos estudar um trecho didático com três sensores e apenas um trem, com um único ímã detectável e sensores separados ao longo do percurso. A circulação ocorre em sentido único, sem paradas com reversão ou retirada manual da composição.

A ordem prevista é S10, depois S11 e depois S12. Esses nomes são exemplos de sensores ligados à gaveta GA3. A sequência deste capítulo não define nem modifica uma rota real da Ferrovia EFMN 74.



Evento	Informação registrada
S10 acionado primeiro	Passagem detectada em S10; etapa estimada rumo a S11.
S11 acionado depois de S10	Passagem detectada em S11; etapa estimada rumo a S12.
S12 acionado depois de S11	Passagem detectada em S12; sequência concluída.
Outro evento fora da ordem	Sequência inconsistente; posição estimada desconhecida.

“Sequência concluída” significa que os três eventos foram reconhecidos na ordem esperada. Não significa que todo o trem deixou o último trecho.

3. Estado inicial: posição desconhecida

Ao ligar a ESP32, o programa ainda não possui um histórico válido. Portanto, começa com a posição desconhecida e aguarda S10 como primeiro ponto de referência.

Se S11 ou S12 for detectado antes de S10, o programa não deve inventar uma passagem anterior. Ele informa uma inconsistência e permanece sem uma estimativa válida.

Quando houver retirada manual do trem, mudança de sentido, perda de uma passagem ou reinicialização da ESP32, o acompanhamento precisa ser revisto antes de continuar.

4. Guardando a etapa do acompanhamento

Vamos utilizar uma variável chamada stage para guardar a etapa reconhecida. Um tipo enum permite dar nomes aos seus possíveis valores:

```
enum TrackingStage {  
    UNKNOWN,  
    AFTER_S10,  
    AFTER_S11,  
    AT_S12,  
    INCONSISTENT  
};
```

```
TrackingStage stage = UNKNOWN;
```



Nome no código	Significado
UNKNOWN	Sem referência inicial válida.
AFTER_S10	Passagem em S10 reconhecida; aguardando S11.
AFTER_S11	Passagem em S11 reconhecida; aguardando S12.
AT_S12	Última passagem reconhecida em S12.
INCONSISTENT	A sequência apresentou um evento inesperado.

Os nomes representam estados do programa. AFTER_S10 não garante que a composição inteira já tenha passado por S10.

5. Do nível elétrico ao evento de passagem

No capítulo 5, guardamos o estado de cada sensor. Para acompanhar o trem, vamos utilizar apenas a mudança de não acionado para acionado.

Assim, um ímã parado sobre o sensor gera um único evento reconhecido, em vez de várias passagens iguais. O retorno para não acionado prepara a entrada para um próximo acionamento.

```
if (stableStates[index]) {  
    registerPassage(sensorIds[index]);  
}
```

Esse trecho deve ser chamado somente quando o estado aceito mudar para acionado. Não o coloque diretamente em um loop() que apenas verifica se o sensor continua ativo.

Se o ímã sair e voltar à área sensível, poderá produzir outro evento. O modelo deste capítulo trata uma repetição inesperada como inconsistência, em vez de avançar a localização sem evidência.

6. Simular antes de ligar os sensores

Para compreender a lógica, o exercício será feito pelo Monitor Serial. Três comandos representarão eventos de passagem:

Comando	Evento simulado
1	Passagem por S10.
2	Passagem por S11.
3	Passagem por S12.
R ou r	Reiniciar o acompanhamento com posição desconhecida.



Utilize uma ESP32 de bancada conectada por USB, sem sensores ou servos. Cada comando enviado representa um novo evento; ele não representa a permanência do ímã sobre o sensor.

7. Programa completo

Crie um programa chamado `intermediate_06_train_tracking` e copie o código abaixo. Não é necessária biblioteca adicional.

```
enum TrackingStage {
    UNKNOWN, AFTER_S10, AFTER_S11, AT_S12, INCONSISTENT
};
TrackingStage stage = UNKNOWN;

void resetTracking() {
    stage = UNKNOWN;
    Serial.println("Position unknown - waiting for S10");
}

void registerPassage(int sensorId) {
    Serial.print("Passage at S");
    Serial.println(sensorId);
    if (stage == INCONSISTENT) {
        Serial.println("Tracking blocked - check and reset");
        return;
    }
    if (sensorId == 10 && stage == UNKNOWN) {
        stage = AFTER_S10;
        Serial.println("Last point: S10 - expected next: S11");
    } else if (sensorId == 11 && stage == AFTER_S10) {
        stage = AFTER_S11;
        Serial.println("Last point: S11 - expected next: S12");
    } else if (sensorId == 12 && stage == AFTER_S11) {
        stage = AT_S12;
        Serial.println("Last point: S12 - sequence completed");
    } else {
        stage = INCONSISTENT;
        Serial.println("Unexpected passage - position unknown");
    }
}
```



Programa completo - continuação

```
void setup() {
  Serial.begin(115200);
  delay(1000);
  Serial.println("EFMN 74 - Single train tracking simulation");
  resetTracking();
  Serial.println("S10: 1 | S11: 2 | S12: 3 | Reset: R");
}

void loop() {
  if (Serial.available() > 0) {
    char command = Serial.read();
    if (command == '1') registerPassage(10);
    else if (command == '2') registerPassage(11);
    else if (command == '3') registerPassage(12);
    else if (command == 'R' || command == 'r') resetTracking();
    else if (command != '\n' && command != '\r') {
      Serial.println("Invalid command");
    }
  }
}
```

8. Como o programa decide

registerPassage() recebe a identificação do sensor e compara o evento com a etapa atual. Somente a combinação esperada permite avançar.

Depois de S10, o próximo evento esperado é S11. Depois de S11, o próximo é S12. Qualquer outra combinação torna o acompanhamento inconsistente.

Quando o estado é INCONSISTENT, novos eventos continuam sendo mostrados, mas não recuperam automaticamente uma posição válida. É necessário conferir a situação e reiniciar o acompanhamento.

O comando R apaga o histórico lógico. Ele não localiza o trem nem libera um trecho. Uma nova sequência válida deverá começar por S10.

Exercício prático

Meta: acompanhar uma sequência correta e observar como o programa reage a eventos inesperados.

1. Conecte a ESP32 de bancada por USB e selecione a placa e a porta.
2. Compile, envie o programa e abra o Monitor Serial em 115200 baud.
3. Pressione EN ou RESET e confira a informação de posição desconhecida.
4. Envie 1, depois 2 e depois 3, um comando de cada vez.
5. Confira o último ponto e o próximo sensor esperado a cada etapa.
6. Envie R para iniciar um novo teste.
7. Envie 2 antes de 1. Confira a inconsistência.



8. Envie 1 enquanto o acompanhamento estiver bloqueado. Confira que ele não é recuperado automaticamente.

9. Envie R e teste a sequência 1, 3. O salto de S11 deverá ser identificado.

10. Envie R e teste 1, 1. O evento repetido também deverá ser identificado como inesperado.

Resultado da sequência correta:

```
Passage at S10
Last point: S10 - expected next: S11
Passage at S11
Last point: S11 - expected next: S12
Passage at S12
Last point: S12 - sequence completed
```

Resultado de um primeiro evento em S11:

```
Passage at S11
Unexpected passage - position unknown
```

Ao terminar uma sequência em S12, utilize R antes de simular outra circulação. O exemplo não possui retorno automático ao ponto inicial.

9. Integração com o programa de sensores

Depois de concluir a simulação, a função `registerPassage()` poderá receber os eventos das entradas do capítulo 5. Faça a integração em um projeto separado, preservando os testes anteriores.

1. Inclua o tipo `TrackingStage`, a variável `stage` e as funções de acompanhamento no programa de sensores.

2. Na função `readSensors()`, localize o trecho que aceita uma mudança e atualiza `stableStates[index]`.

3. Mantenha a mensagem de mudança e, quando o novo estado for acionado, chame `registerPassage(sensorIds[index]);`.

4. Na inicialização, comece com posição desconhecida e confira os sensores antes de iniciar a circulação.

O trecho de reconhecimento de mudança ficará assim:

```
stableStates[index] = active;
showSensorState(index);
if (active) {
    registerPassage(sensorIds[index]);
}
```

Mantenha as listas, as ligações e a verificação de estabilidade do capítulo 5. Não copie dois `setup()` ou dois `loop()` no mesmo projeto: combine suas tarefas nas funções principais existentes.

Se o trem já estiver sobre um sensor quando o sistema for ligado, a leitura inicial não reconstrói sua passagem anterior. Posicione a composição antes de S10, com as entradas inicialmente não acionadas, para iniciar este exercício em condições conhecidas.



10. Limites do acompanhamento

Situação	Limite deste modelo
Mais de um trem no trecho	Os eventos não identificam qual trem passou.
Circulação em sentido contrário	A sequência prevista deixa de corresponder aos eventos.
Vários ímãs na mesma composição	Podem surgir vários acionamentos para uma única passagem.
Uma passagem perdida	A próxima passagem pode produzir uma inconsistência; se não houver outra, a estimativa fica desatualizada.
Trem parado entre sensores	O último evento não informa uma coordenada nem comprova movimento.
Fim da composição ainda no trecho	A passagem do ímã da locomotiva não confirma a liberação do trecho.

Para reconhecer o sentido, podemos utilizar uma sequência adequada de pontos de detecção. Para acompanhar vários trens ou liberar trechos, serão necessárias informações e regras adicionais.

Não use a última posição estimada, sozinha, para autorizar o movimento de um desvio ou liberar um sinal. O acompanhamento deste capítulo não constitui a lógica completa de segurança da ferrovia.

Resumo do capítulo

- O sensor fornece uma detecção pontual.
- O histórico de passagens permite construir uma localização estimada.
- O programa começa com posição desconhecida.
- Uma sequência válida depende das condições de operação previstas.
- Eventos inesperados devem interromper a confiança na estimativa.
- A permanência do ímã não deve gerar eventos repetidos a cada leitura.
- Reiniciar o acompanhamento não localiza o trem fisicamente.
- Passagem detectada e trecho livre são informações diferentes.

Próximo capítulo: Controle de sinais ferroviários.